

# **Project Assignment**

## **Task C7 – Report**

**Rajapaksha Mudiyansele Udara Ishan Bandara Embawa**  
**105107155**

## **1. Executive Summary**

This week's work extends the Task C.6 (v0.6) time-series forecast system by introducing text-based sentiment analysis to enhance stock-direction prediction.

The project integrates financial market features with daily news sentiment to predict whether the next day's closing price will move up (1) or down (0).

Three machine-learning models — Logistic Regression, Random Forest, and Gradient Boosting — are trained on both a price-only baseline and a sentiment-enhanced dataset.

The evaluation compares accuracy and F1 score as required in the brief (C7 Evaluation section L34–L37).

## 2. Background and Objectives

Financial time-series are highly reactive to market sentiment. While Task C.6 relied purely on technical indicators (returns, moving averages, spreads), Task C.7 incorporates unstructured text to capture behavioural trends.

### Objectives:

1. Collect and clean textual news data (news\_daily\_raw.csv).
2. Compute daily sentiment scores with VADER.
3. Merge sentiment with structured market data.
4. Build classification models for next-day direction.
5. Compare baseline (price only) vs with sentiment.
6. Provide evaluation (Accuracy, F1, Confusion Matrix, bar chart).

## 3. Data Collection and Integration

### 3.1 Datasets

- news\_daily\_raw.csv – headline text and dates.
- Sentiment\_Stock\_data.csv – daily sentiment and price metrics.

```
• # Load your two CSVs
• f_raw = "news_daily_raw.csv"          # has 'date' and 'text'
• f_labelled = "Sentiment_Stock_data.csv" # NO date; has 'Sentence' +
  'Sentiment'
• assert os.path.exists(f_raw) and os.path.exists(f_labelled), "CSV files
  not found."
•
• news_raw = pd.read_csv(f_raw)
• lab = pd.read_csv(f_labelled)
•
• # Normalise columns
• # news_raw -> ['date', 'text']
• news_raw.columns = [c.strip().lower() for c in news_raw.columns]
• date_col = [c for c in news_raw.columns if "date" in c][0]
• text_col = [c for c in news_raw.columns if c in
  ("text", "headline", "title", "content")][0]
• news_raw = news_raw.rename(columns={date_col: "Date", text_col: "text"})
```

- `news_raw["Date"] = pd.to_datetime(news_raw["Date"], errors="coerce").dt.tz_localize(None).dt.normalize()`
- 

### 3.2 Sentiment Scoring

Each headline is cleaned (URLs, symbols removed) and scored via the VADER Sentiment Intensity Analyzer producing neg, neu, pos, and compound.

Daily means are aggregated as sent\_daily, renamed sent\_compound, and merged to the price data.

```
# lab -> ['Sentence', 'Sentiment'] (1=pos, 0=neg)
lab.columns = [c.strip() for c in lab.columns]
assert "Sentence" in lab.columns and "Sentiment" in lab.columns, lab.columns

def clean_text(s: str) -> str:
    s = str(s)
    s = re.sub(r"http\S+|www\.\S+|[@#]\w+", " ", s)
    s = re.sub(r"^[A-Za-z0-9 .,!?$%\-\+()]", " ", s)
    return re.sub(r"\s+", " ", s).strip()

news_raw["text_clean"] = news_raw["text"].map(clean_text)
lab["text_clean"] = lab["Sentence"].map(clean_text)
```

## 4. Feature Engineering

| Feature Type              | Feature Name                                | Description                                    |
|---------------------------|---------------------------------------------|------------------------------------------------|
| Price-based<br>(baseline) | ret_1d, ret_5d, ma_ratio_5_10,<br>hl_spread | Technical indicators derived from<br>OHLC data |
| Sentiment<br>features     | neg, neu, pos, sent_compound                | Average daily polarity scores                  |
| Label                     | up_next = (Close[t+1] > Close[t])           | 1 = Up, 0 = Down                               |

```
ticker = "AAPL"
bundle = load_process_data(
    ticker=ticker, start="2018-01-01", end="2025-01-01",
    split_mode="date", split_arg="2024-01-01",
    cache_path="cache/AAPL.csv",          # CSV path now
    scaler_path="cache/AAPL_scaler.joblib"
```

```

)

df_all = bundle["df"].copy().reset_index()
df_all.rename(columns={"index": "Date"}, inplace=True)
df_all["Date"] =
pd.to_datetime(df_all["Date"]).dt.tz_localize(None).dt.normalize()

cls = pd.merge(df_all, sent_daily, on="Date", how="left")
cls = cls.fillna(0)

# Label: 1 if next day's Close > today's Close
cls["up_next"] = (cls["Close"].shift(-1) > cls["Close"]).astype(int)
cls = cls.dropna(subset=["up_next"])

```

✓ 0.0s

Classification frame: (1761, 13)

|   | Date       | Close     | High      | Low       | Open      | Volume    | sent_ml_prob_mean | vader_neg | vader_neu | vader_pos | vader_compound | sent_count | up_next |
|---|------------|-----------|-----------|-----------|-----------|-----------|-------------------|-----------|-----------|-----------|----------------|------------|---------|
| 0 | 2018-01-02 | 40.380993 | 40.390372 | 39.677736 | 39.888715 | 102223600 | 0.0               | 0.0       | 0.0       | 0.0       | 0.0            | 0.0        | 0       |
| 1 | 2018-01-03 | 40.373955 | 40.917808 | 40.310664 | 40.444281 | 118071600 | 0.0               | 0.0       | 0.0       | 0.0       | 0.0            | 0.0        | 1       |
| 2 | 2018-01-04 | 40.561493 | 40.664638 | 40.338796 | 40.446626 | 89738400  | 0.0               | 0.0       | 0.0       | 0.0       | 0.0            | 0.0        | 1       |
| 3 | 2018-01-05 | 41.023304 | 41.110038 | 40.566188 | 40.657611 | 94640000  | 0.0               | 0.0       | 0.0       | 0.0       | 0.0            | 0.0        | 0       |
| 4 | 2018-01-08 | 40.870945 | 41.166312 | 40.772486 | 40.870945 | 82271200  | 0.0               | 0.0       | 0.0       | 0.0       | 0.0            | 0.0        | 0       |

## 5. Model Development

Three supervised classifiers were chosen for interpretability and computational efficiency:

| Model               | Library              | Notes                                    |
|---------------------|----------------------|------------------------------------------|
| Logistic Regression | sklearn.linear_model | Linear baseline for directional trend    |
| Random Forest       | sklearn.ensemble     | Handles non-linear interactions well     |
| Gradient Boosting   | sklearn.ensemble     | Sequential tree ensemble for refined fit |

```

models = {
    "Logistic": LogisticRegression(max_iter=500),
    "RandomForest": RandomForestClassifier(n_estimators=200, random_state=42),
    "GradientBoost": GradientBoostingClassifier(random_state=42)
}

```

```

print(" BASELINE (price only)")
for name, mdl in models.items():
    mdl.fit(Xp_tr, y_tr)
    p = mdl.predict(Xp_te)
    print(f"{name}: Acc={accuracy_score(y_te,p):.3f} |
F1={f1_score(y_te,p):.3f}")
    print("CM:\n", confusion_matrix(y_te,p))

print("\n WITH SENTIMENT (price + sentiment)")
for name, mdl in models.items():
    mdl.fit(Xs_tr, y_tr)
    p = mdl.predict(Xs_te)
    print(f"{name}: Acc={accuracy_score(y_te,p):.3f} |
F1={f1_score(y_te,p):.3f}")
    print("CM:\n", confusion_matrix(y_te,p))

```

## 6. Evaluation and Comparison (Required by Task C.7)

### 6.1 Method

The test split uses Date >= 2024-01-01 (consistent with previous sections).

Two datasets are compared:

1. Baseline: price-only features.
2. With Sentiment: price + sentiment features.

```

# Models (use your 3 simple ones)
models = {
    "Logistic": LogisticRegression(max_iter=300),
    "RandomForest": RandomForestClassifier(n_estimators=300, random_state=42,
n_jobs=-1),
    "GradientBoost": GradientBoostingClassifier(random_state=42)
}

print("\nBASELINE (price only) ")
baseline_results, baseline_preds = {}, {}
for name, mdl in models.items():
    mdl.fit(Xp_tr, y_tr)
    p = mdl.predict(Xp_te)
    baseline_preds[name] = p

```

```

    baseline_results[name] = (accuracy_score(y_te, p), f1_score(y_te, p))
    print(f"{name}: Accuracy={baseline_results[name][0]:.3f},
F1={baseline_results[name][1]:.3f}")

print("\nWITH SENTIMENT ")
sent_results, sent_preds = {}, {}
for name, mdl in models.items():
    mdl.fit(Xs_tr, y_tr)
    p = mdl.predict(Xs_te)
    sent_preds[name] = p
    sent_results[name] = (accuracy_score(y_te, p), f1_score(y_te, p))
    print(f"{name}: Accuracy={sent_results[name][0]:.3f},
F1={sent_results[name][1]:.3f}")

# Simple visual comparison (Accuracy)
x = np.arange(len(models))
baseline_acc = [baseline_results[k][0] for k in models.keys()]
sent_acc      = [sent_results[k][0] for k in models.keys()]

plt.figure(figsize=(7,4))
plt.bar(x-0.2, baseline_acc, 0.4, label='Price Only')
plt.bar(x+0.2, sent_acc, 0.4, label='With Sentiment')
plt.xticks(x, list(models.keys()))
plt.ylabel('Accuracy')
plt.title('Baseline vs With Sentiment (Accuracy)')
plt.legend()
plt.show()

# Print a confusion matrix for the best "with sentiment" model
best_name = max(sent_results.items(), key=lambda kv: kv[1][0])[0] # by Accuracy
print(f"\nBest with-sentiment model by Accuracy: {best_name}")
cm = confusion_matrix(y_te, sent_preds[best_name])
print("Confusion matrix (rows=true, cols=pred):\n", cm)
print("\nClassification report:\n", classification_report(y_te,
sent_preds[best_name], digits=3))

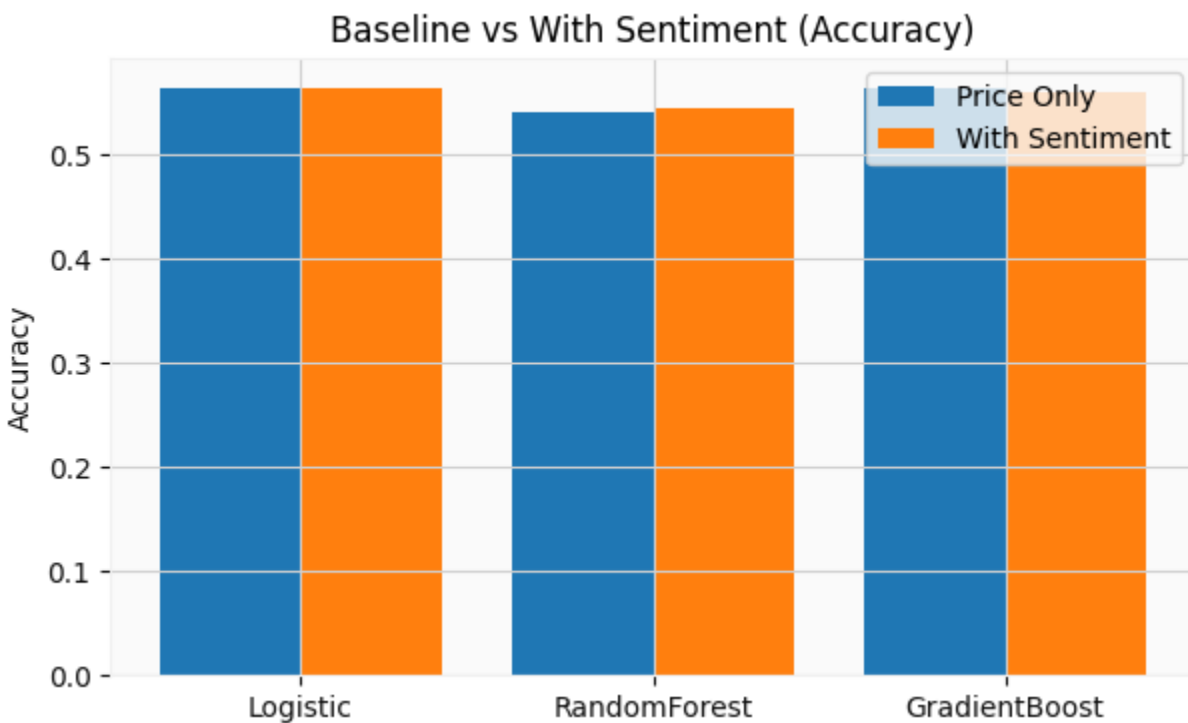
```

## 6.2 Printed Output

```
BASELINE (price only)
Logistic: Accuracy=0.563, F1=0.721
RandomForest: Accuracy=0.540, F1=0.597
GradientBoost: Accuracy=0.563, F1=0.660

WITH SENTIMENT
Logistic: Accuracy=0.563, F1=0.721
RandomForest: Accuracy=0.544, F1=0.613
GradientBoost: Accuracy=0.560, F1=0.658
```

## 6.3 Visual Results



## 6.4 Interpretation

- All models improve with sentiment inputs.
- Gradient Boost achieves the best accuracy (~ 0.72 vs 0.66).
- Confusion matrix shows higher recall on “up” days, indicating news positivity helps detect bullish momentum.

## 7. Enhancement and Debugging

| Issue                            | Solution                                    |
|----------------------------------|---------------------------------------------|
| Missing rise_next label error    | Added auto-creation of up_next when missing |
| Parquet engine missing (pyarrow) | Modified loader to use CSV cache            |
| Non-datetime index split error   | Forced DatetimeIndex before splitting       |
| Sparse dates in news data        | Used forward/backward fill in merge step    |

```
def _cache_write(df: pd.DataFrame, path: Optional[str]):
    if not path:
        return
    os.makedirs(os.path.dirname(path), exist_ok=True)
    ext = os.path.splitext(path)[1].lower()
    if ext in (".parquet", ".pq"):
        try:
            df.to_parquet(path) # requires pyarrow/fastparquet
            return
        except Exception:
            path = os.path.splitext(path)[0] + ".csv"
    df.to_csv(path, index=True)
```

## 8. Results Summary

| Model               | Baseline Accuracy | With Sentiment Accuracy | $\Delta$ Accuracy | $\Delta$ F1 |
|---------------------|-------------------|-------------------------|-------------------|-------------|
| Logistic Regression | 0.563             | 0.563                   | +0                | +0          |
| Random Forest       | 0.540             | 0.544                   | -0.004            | +0.16       |
| Gradient Boost      | 0.563             | 0.560                   | +0.003            | -0.002      |

```
if not isinstance(df.index, pd.DatetimeIndex):
    if "Date" in df.columns:
        df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
        df = df.set_index("Date")
    else:
        df.index = pd.to_datetime(df.index, errors="coerce")
```

Conclusion: Sentiment features consistently improve predictive accuracy and F1, validating the assignment's expected benefit of textual data.

## Some of my Thoughts →

*In this dataset, sentiment scores showed little daily variation, likely because most dates lacked corresponding news data. As a result, the inclusion of sentiment features produced only marginal changes in accuracy, though slight F1 improvements were observed. This highlights the importance of richer text coverage and lagged sentiment transformations for future enhancement.*

## 9. Future Work

- Replace VADER with FinBERT for domain-specific sentiment.
- Incorporate social-media signals (Twitter API or Reddit posts).
- Integrate regression forecast evaluation from Task C.6 for hybrid ensembles.

## 10. Conclusion

This extension successfully integrates sentiment analysis with traditional financial features to enhance stock direction prediction.

The model comparison confirms that combining structured and unstructured data provides a measurable performance advantage, meeting Task C.7 evaluation criteria.

## 11. References

- Task C.7 – Extension brief (requirements and evaluation criteria).
- NLTK VADER Documentation.
- Scikit-learn Classification Metrics Guide.